Final Project Simulation Specification (ROS2)

TurtleBot3 "Vacuum Cleaner" in Gazebo House World

Course: _____ Term: _____ Instructor: _____

1. Project Summary

You will build an **application-level autonomy layer** on top of the existing TurtleBot3 ROS2 packages for **SLAM** and **Navigation (Nav2)** in the **Gazebo House (home) environment**. The robot should behave like a simple vacuum cleaner: it can navigate to rooms by name and clean rooms on request, while avoiding obstacles.

2. Environment and Allowed Tools

- Platform: **TurtleBot3** in **Gazebo** using the provided **House/Home** world.
- You may use existing ROS2 packages for:
  - **SLAM** (map creation),
  - **Nav2** (localization and goal navigation).
- You must create your own **new ROS2 package** that contains your project logic (nodes, launch files, configs, and documentation).

3. Functional Requirements (What your system must do)

3.1 Mapping Phase (Required)

- Create a map of the environment using SLAM as practiced in class.
- Provide clear instructions to reproduce the mapping step (or to load a saved map).

3.2 Room Specification (Required)

Students must define rooms with:

- **Room names** (e.g., kitchen, bedroom, living_room).
- **Room boundaries** so the robot knows which area belongs to each room.

How to specify room boundaries:

- Each room must be described by a set of user-provided boundary points in the **map frame**.
- A simple minimum option is a rectangle (4 points). Other representations are allowed if documented.
- The room definitions must be reproducible (same names and boundaries every run).

3.3 Command Interface (Required)

Your system must support **two commands**:

1. **Navigate to room by name**: e.g., goto_room("kitchen")
2. **Clean room by name**: e.g., clean_room("bedroom")

The commands may be implemented as:

- ROS2 services, actions, or topics; or
- a CLI node that accepts room names as input parameters.

Important: In the final demo, the robot must navigate **autonomously**. Using RViz "click-to-goal" as the main method is **not allowed** for the final solution.

3.4 Navigation to a Named Room (Required)

- Given a room name, the system selects an appropriate target pose inside that room and sends the goal to Nav2 programmatically.
- The robot must reach the room reliably (within reasonable time and without collisions).

3.5 Cleaning a Named Room (Required)

Cleaning means executing a motion strategy inside the room boundary to maximize floor coverage.

Two difficulty options:

- **Baseline (required for passing):** Students specify a set of waypoints inside each room (manual waypoints).
- **Advanced (optional/bonus):** The program automatically generates cleaning waypoints for a room (e.g., lane/grid pattern).

4. Obstacle Handling Requirements

4.1 Static Obstacles (Required)

- The robot must avoid walls and furniture (static obstacles).
- Students may tune Nav2 and costmap parameters to improve safety and performance.

4.2 Dynamic Obstacles (Required behavior + optional bonus)

Required baseline behavior:

- During cleaning, if a waypoint goal fails (e.g., blocked temporarily), the system should **skip** it (or retry once and then skip) and continue cleaning remaining points.

Optional/bonus behavior:

- Smarter dynamic-obstacle handling (e.g., postpone blocked points and revisit later, adapt the plan).

5. Coverage Metric and Testing

- **Room coverage (%) is an important evaluation metric.**
- **Students do not need to implement coverage calculation.** The instructor will compute coverage using an external test method.
- Students should design strategies that balance:
  - **coverage** (higher is better),
  - **time to clean** (faster is better),
  - **robustness** (fewer failures, safe behavior).

6. Deliverables (What you must submit)

1. **ROS2 Package** (source code) containing:
   - nodes implementing the two commands (goto_room and clean_room),
   - launch files,
   - configuration files (room names/boundaries; manual waypoints if used),
   - a clear README with step-by-step run instructions.
2. **Demo** (live or recorded, as instructed) showing:
   - mapping (or loading a saved map),
   - navigating to at least one named room,
   - cleaning at least one named room autonomously,
   - behavior when at least one cleaning waypoint fails (skip / retry then skip).
3. **Short explanation** (report or slides if requested) covering:
   - how rooms are represented,
   - cleaning strategy (manual or automatic),
   - how goals are sent to Nav2,
   - any parameter tuning and its effect.

7. Grading Rubric (Core Points)

Core total: 100 points. (Instructor may adjust weights.)

1. **Reproducibility and Packaging (20 pts)**
   - Package builds and runs reliably (10)
   - Clear README and reproducible setup (10)
2. **Room Definition and Interface (15 pts)**
   - Correct room names + boundaries in map frame (10)
   - Commands accept room names and behave consistently (5)
3. **Go-To-Room by Name (20 pts)**
   - Programmatically sends Nav2 goals (no RViz clicking) (10)
   - Reaches target room(s) reliably and safely (10)
4. **Clean-Room Behavior (30 pts)**
   - Systematic cleaning strategy within room boundary (15)
   - Good performance: time/coverage trade-off (instructor measured) (10)
   - Failure handling during cleaning (skip / retry-then-skip) (5)
5. **Robustness and Safety (15 pts)**
   - Avoids static obstacles safely (10)
   - Handles blocked/failed waypoints gracefully (5)

8. Bonus Points (Optional Advanced Features)

Up to +20 bonus points (instructor may cap the final grade).

- **Automatic Waypoint Generation (+8 to +12)**
  - Generate lane/grid waypoints inside the room boundary
  - Remove unsafe waypoints near obstacles (based on robot size / clearance)
- **Advanced Dynamic Obstacle Handling (+4 to +8)**
  - Adapt cleaning plan after repeated failures (postpone/revisit/remove region)
- **Navigation/Costmap Optimization (+2 to +6)**
  - Demonstrate measurable improvements (faster cleaning, fewer failures, smoother motion)

  - Explain which parameters changed and why

9. Rules and Notes

- SLAM and Nav2 packages are allowed and expected; your grade is primarily based on your **custom autonomy layer**.
- Manual RViz goal clicking is not acceptable as the final solution.
- Coverage (%) is evaluated by the instructor externally; you do not need to implement coverage computation.
- Advanced features are optional and give bonus points; they are not required to pass.

Instructor Notes (optional): This document is intended to be used directly as the student-facing project specification and as a basis for grading rubric and course planning.